

Accelerated Test-Time Scaling with Model-Free Speculative Sampling

Anonymous Authors¹

Abstract

Language models have demonstrated remarkable capabilities in reasoning tasks through test-time scaling techniques like best-of-N sampling and tree search. However, these approaches often demand substantial computational resources, creating a critical trade-off between performance and efficiency. We introduce STAND (STochastic Adaptive N-gram Drafting), a novel model-free speculative decoding approach that leverages the inherent redundancy in reasoning trajectories to achieve significant acceleration without compromising accuracy. Our analysis reveals that reasoning paths frequently reuse similar reasoning patterns, enabling efficient model-free token prediction without requiring separate draft models. By introducing stochastic drafting and preserving probabilistic information through a memory-efficient logit-based N-gram module, combined with optimized Gumbel-Top-K sampling and data-driven tree construction, STAND significantly improves token acceptance rates. Extensive evaluations across multiple models and reasoning tasks (AIME-2024, GPQA-Diamond, and LiveCodeBench) demonstrate that STAND reduces inference latency by 60-65% compared to standard autoregressive decoding while maintaining accuracy. Furthermore, STAND outperforms state-of-the-art speculative decoding methods by 14-28% in throughput and shows strong performance even in single-trajectory scenarios, reducing inference latency by 48-58%. As a model-free approach, STAND can be applied to any existing language model without additional training, being a powerful plug-and-play solution for accelerating language model reasoning.

¹ Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

1. Introduction

Test-time scaling has emerged as a prominent paradigm for enhancing the performance of language models by allocating additional computational resources during inference (Snell et al., 2024). This includes generating long sequences of thoughts through Large Reasoning Models (LRMs) (Muennighoff et al., 2025), multi-sampling approaches like best-of-N sampling and majority voting that generate multiple independent outputs to select the most promising one (Wang et al., 2022), as well as iterative methods like tree search and sequential refinement that allow models to progressively improve their reasoning (Uesato et al., 2022). While these methods demonstrate the potential for significant accuracy improvements, they often demand substantial computational resources due to the large number of tokens that need to be generated.

Recent research has focused on reducing the high computational costs of test-time scaling and reasoning approaches (Sui et al., 2025). Some work has explored training with length-based rewards to generate more concise outputs (Aggarwal & Welleck, 2025; Qu et al., 2025), while other approaches use combinations of small and large models to distribute the workload efficiently (Liao et al., 2025; Yang et al., 2025). However, these efficiency-focused methods typically face a fundamental trade-off. While they reduce computational costs, they tend to sacrifice some accuracy compared to more exhaustive approaches, as using fewer samples or cutting short the exploration process often leads to lower performance. This raises a crucial question:

How can we improve the efficiency of test-time scaling and reasoning approaches without compromising their accuracy?

To address this challenge, we turn our attention to speculative decoding (SD), which offers a promising solution for lossless acceleration of language model inference. Speculative decoding accelerates language model inference by using a smaller "draft" model to predict tokens, which are then verified by the larger target model (Leviathan et al., 2023). With appropriate verification strategies like speculative sampling (Chen et al., 2023a), SD can speed up the auto-regressive decoding process of large language models while preserving their original output distribution.

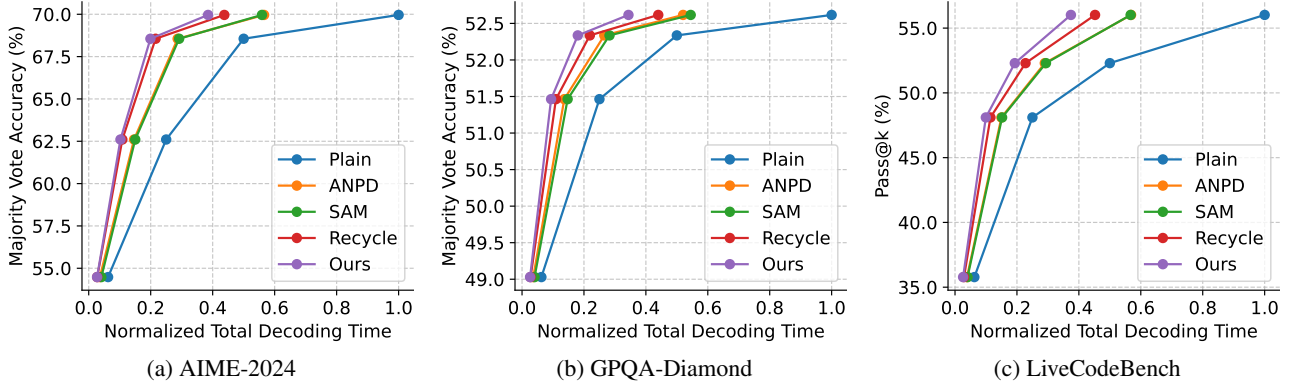


Figure 1. Scaling curve with speculative decoding. We report the scaling curve describing how the task performance improves with respect to the total decoding time. Keeping simple auto-regressive decoding total time as 1, we also report the scaling curves for different model-free SD methods. We report the reward-weighted majority voting accuracy for AIME-2024 and GPQA-Diamond, and pass@k for LiveCodeBench, where k is the total number of generated sequences generated at a given point. All measurements are made on a single A100 GPU with DeepSeek-R1-Distill-Qwen-7B.

A key observation in LRMs is the significant repetition of token sequences across different reasoning paths. When models are performing chain-of-thought reasoning (Snell et al., 2024) or exploring multiple solutions (Wang et al., 2022; Xie et al., 2024), they frequently reuse similar expressions, logical deductions, and reasoning patterns.

This redundancy presents an opportunity for model-free speculative decoding (Saxena, 2023; Ou et al., 2024). Unlike model-based approaches that rely on neural networks as drafters (Li et al., 2024c; Cai et al., 2024), model-free methods can leverage patterns from previous generations to construct drafts. This makes them particularly well-suited for exploiting cross-trajectory information. Our experiments confirm this approach’s effectiveness, demonstrating improved efficiency as the number of reasoning trajectories increases.

To fully leverage the power of model-free speculative decoding for reasoning tasks, we propose **STAND (STochastic Adaptive N-gram Drafting)**. Our approach is motivated by two key observations: First, existing model-free approaches have primarily focused on greedy decoding, leaving the potential benefits of sampling largely unexplored. Second, our experimental analysis demonstrates that stochastic drafting (i.e. sampling draft tokens from the draft probability distribution) significantly improves token acceptance rates. Building on these insights, STAND introduces three key innovations: (1) a memory-efficient logit-based N-gram module that preserves probabilistic information for better stochastic drafting, (2) an optimized sampling strategy using Gumbel-Top-K for efficient token selection, and (3) a data-driven approach to draft tree construction that balances efficiency with effectiveness. Combined, these techniques significantly enhance the speculative decoding performance in the context of test-time scaling, where sampling and di-

verse trajectory exploration are crucial.

Our extensive evaluations demonstrate STAND’s effectiveness in various reasoning tasks (math, science and coding) and different model scales. As highlighted in Figure 1, our results show that STAND’s benefits become more pronounced as the number of reasoning trajectories increases. When using best-of-16 sampling to achieve optimal accuracy, STAND reduces inference latency by 60-65% compared to standard autoregressive decoding while maintaining the same performance level. Moreover, STAND outperforms state-of-the-art speculative decoding methods by 14-28% in throughput, establishing an efficient drafting strategy for reasoning tasks.

Furthermore, we observe that STAND also shows the best throughput in single-trajectory evaluations, reducing the inference latency by 48-58% compared to standard autoregressive decoding, although it was primarily designed to leverage information across multiple reasoning trajectories. As a model-free speculative decoding approach, STAND accomplishes all these achievements without requiring any additional drafter model, or fine-tuning the target model, being able to be used in plug-and-play manner to any existing LRMs.

2. STAND

In this section, we present the details of STAND. In Section 2.1, we propose a memory- and compute-efficient approach to construct the logit-based N-gram module. Then in Section 2.2, we illustrate how to use the N-gram module as a drafter for stochastic sampling, together with several optimization techniques that further improve performance.

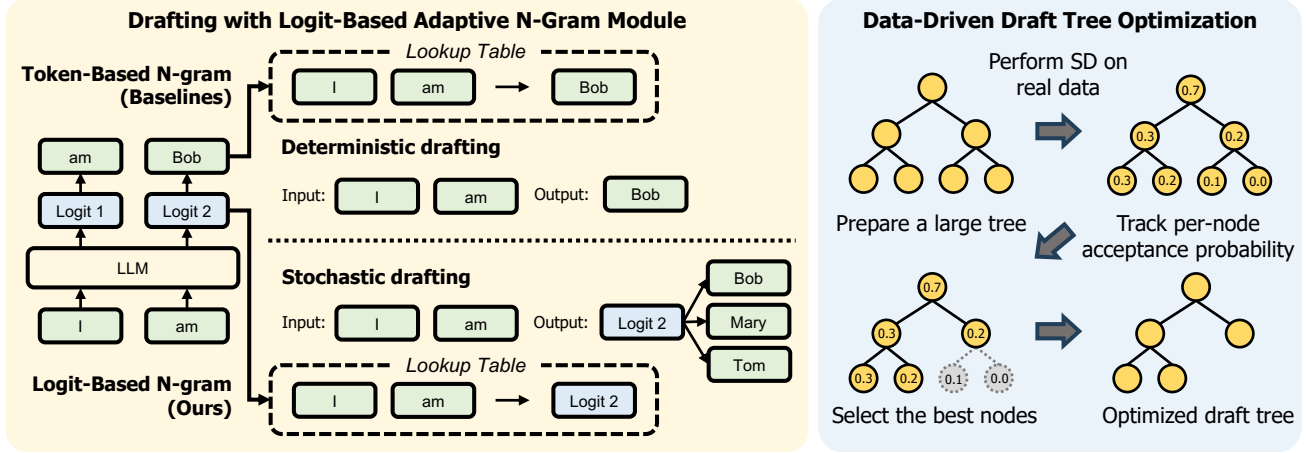


Figure 2. **Overview of STAND.** (Left) The N-gram module of STAND stores logits instead of discrete tokens, enabling stochastic drafting. When the language model generates “I am Bob”, we store the probability distribution over the next token rather than just the sampled token. (Right) Data-driven draft tree optimization: We start with an initial large draft tree, measure node-wise acceptance rates during speculative decoding on real data, and prune to retain the most successful paths.

2.1. Logit-based adaptive N-gram module

Traditional N-gram modules for speculative decoding typically store pairs of N-grams and their corresponding next tokens (Ou et al., 2024). We improve this approach by instead storing the logit distribution from which the next token is sampled. This modification preserves the rich probabilistic information of potential next tokens, enabling more sophisticated stochastic drafting strategies. While existing methods like Token Recycle partially utilize logit information by storing top-k token IDs, they discard valuable probability information that are crucial for stochastic drafting. Like previous works (Saxena, 2023; Ou et al., 2024), we maintain separate lookup tables from unigrams to 4-grams.

Efficient logit approximation. To address the memory overhead associated with storing full logit distributions, particularly for models with large vocabularies, we implement a compressed representation scheme. Our approach maintains only the top-k indices and their corresponding probabilities. When encountering repeated n-grams, we merge distributions by treating non-stored indices as having zero probability and computing a weighted average: for an n-gram seen k times previously, the existing distribution (representing the mean of k occurrences) is weighted by $k/(k+1)$ and the new distribution by $1/(k+1)$. The resulting averaged distribution is then truncated to retain only the top-10 most probable tokens, ensuring constant memory usage while preserving the most relevant probability information for future speculation.

2.2. Drafting with STAND

Stochastic tree drafting. For each position in the draft tree, we predict the next tokens using a multi-level N-gram

approach. Following previous works (Saxena, 2023; Ou et al., 2024), we search for matching N-grams in decreasing order of length, from 4-grams down to unigrams, using the first successful match. This lookup returns the top-10 candidate tokens and their corresponding probabilities from our stored distributions. Based on the number of children required at each tree node, we sample k tokens without replacement from these candidates. These sampled tokens then undergo standard speculative sampling verification to ensure draft quality.

Gumbel-Top-K sampling. For efficient stochastic drafting, we replace sequential sampling with a parallel sampling approach based on the Gumbel-Top-K trick (Kool et al., 2019). For each candidate token’s log probability ϕ_i , we add Gumbel noise to create a perturbed distribution:

$$\phi'_i = \phi_i - \log(-\log U_i), \quad U_i \sim \text{Uniform}(0, 1)$$

Taking the top-k indices from these perturbed values ϕ'_i effectively samples k tokens without replacement in parallel, significantly reducing sampling latency compared to sequential methods.

To further optimize performance, we pre-compute and cache the Gumbel noise terms rather than generating them during drafting. This cached noise is periodically refreshed when depleted, effectively separating the sampling overhead from drafting. These optimizations further enhance the performance of our stochastic drafting approach.

Draft tree optimization. Tree-based speculative decoding typically uses either dynamic trees constructed during inference or static trees built using heuristics. While dynamic trees offer context-adaptability, they add computa-

Table 1. Speculative decoding performance in multi-trajectory reasoning. We report the average throughput (T) and acceptance length (A) for multi-trajectory test-time scaling scenarios, with different number of reasoning trajectories per problem. We evaluate each model on AIME-2024 (AIME), GPQA-Diamond (GPQA), and LiveCodeBench (LCB). The best values are highlighted in **bold**.

		4 Trajectories			8 Trajectories			16 Trajectories		
		AIME	GPQA	LCB	AIME	GPQA	LCB	AIME	GPQA	LCB
<i>DeepSeek-R1-Distill-Qwen-7B</i>										
Plain	T	26.63	31.34	27.75	26.63	31.34	27.75	26.63	31.34	27.75
	A	2.21	1.99	2.13	2.21	1.99	2.13	2.21	1.99	2.13
Eagle-2	T	29.91	31.69	27.61	29.91	31.69	27.61	29.91	31.69	27.61
	A	2.21	1.99	2.13	2.21	1.99	2.13	2.21	1.99	2.13
PLD	T	43.93	50.49	44.01	44.95	53.04	45.08	46.60	53.47	46.02
	A	1.78	1.81	1.73	1.84	1.89	1.79	1.89	1.96	1.85
ANPD	T	45.52	57.39	46.30	46.40	58.97	47.86	47.06	60.25	48.81
	A	1.89	1.97	1.88	1.92	2.03	1.91	1.96	2.11	1.96
SAM	T	44.35	53.21	45.63	45.64	55.47	47.24	47.64	57.53	48.92
	A	1.81	1.87	1.85	1.89	1.96	1.89	1.97	2.03	1.95
Recycle	T	61.38	71.51	60.62	61.70	71.55	60.93	60.86	71.23	61.36
	A	2.76	2.73	2.73	2.77	2.73	2.73	2.77	2.73	2.74
STAND (Ours)	T	64.99	83.47	69.70	66.88	87.02	71.83	69.15	91.17	74.14
	A	3.21	3.48	3.30	3.35	3.70	3.47	3.46	3.90	3.64

tional overhead. Conversely, static trees are computationally efficient but may underperform if constructed through heuristics alone.

We address this limitation through a data-driven approach to static tree construction. Our method begins by initializing a large tree with 625 nodes and performing speculative decoding on 30 samples from the AIME-2024 dataset. During this process, we track which nodes are frequently part of successful speculation paths. We then select the top-80 most effective nodes and reorganize them into a compact tree structure. This empirical approach maintains the computational efficiency of static trees while ensuring the tree structure is optimized based on real-world performance data.

3. Experiments

In this section, we highlight the effectiveness of STAND through extensive evaluations. Specifically, we evaluate STAND’s performance in multi-trajectory inference in Figure 1 and Table 1, where we generate multiple candidate answers by sequentially producing k independent reasoning traces and then aggregate the results.

As shown in Figure 1, STAND significantly improves decoding efficiency, achieving equivalent performance to plain decoding in less than 40% the time. Table 1 provides detailed throughput and acceptance length comparisons across methods. STAND not only achieves the highest throughput but also maintains longer acceptance lengths compared to baselines. Importantly, both metrics improve as we increase the number of trajectories, making STAND’s speedup ad-

vantage more pronounced with increased compute scaling.

Notably, Token Recycle’s performance does not improve with more trajectories, unlike other model-free approaches. This limitation likely comes from its lookup table update strategy, which replaces rather than aggregates information from new trajectories. While this approach may offer drafting speed benefits, STAND’s superior and scaling-dependent performance suggests that aggregating historical information is beneficial for test-time scaling.

We provide further experiments and analysis in Section C. Specifically, in Section C.1, we further extend the results in Table 1 with more baseline and models, further showcasing the effectiveness of STAND in multi-trajectory reasoning. In Section C.2, we showcase that STAND is also effective for single-trajectory reasoning, where we generate only a single reasoning chain. Finally in Section C.3, we provide additional ablations and analysis to showcase the effectiveness of individual components of STAND.

4. Conclusion

In this work, we introduced STAND, a model-free speculative decoding approach that accelerates language model reasoning while maintaining accuracy. By utilizing reasoning trajectory redundancy and historical logit information, STAND significantly improves throughput over standard auto-regressive decoding. Our method outperforms existing alternatives, offering an efficient solution for scaling AI reasoning systems.

References

- Aggarwal, P. and Welleck, S. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023a.
- Chen, X., Aksitov, R., Alon, U., Ren, J., Xiao, K., Yin, P., Prakash, S., Sutton, C., Wang, X., and Zhou, D. Universal self-consistency for large language model generation. *arXiv preprint arXiv:2311.17311*, 2023b.
- Cheng, Y., Zhang, A., Zhang, X., Wang, C., and Wang, Y. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Geva, M., Schuster, T., Berant, J., and Levy, O. Token recycling: Making llms faster and more data-efficient. *arXiv preprint arXiv:2310.02548*, 2023.
- Hu, Y., Wang, K., Zhang, X., Zhang, F., Li, C., Chen, H., and Zhang, J. Sam decoding: Speculative decoding via suffix automaton. *arXiv preprint arXiv:2411.10666*, 2024.
- Hu, Y., Liu, Z., Dong, Z., Peng, T., McDanel, B., and Zhang, S. Q. Speculative decoding and beyond: An in-depth survey of techniques. *arXiv preprint arXiv:2502.19732*, 2025.
- Kool, W., Van Hoof, H., and Welling, M. Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement. In *International Conference on Machine Learning*, pp. 3499–3508. PMLR, 2019.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.
- Li, M., Chen, X., Holtzman, A., Chen, B., Lin, J., Yih, S., and Lin, V. Nearest neighbor speculative decoding for llm generation and attribution. *Advances in Neural Information Processing Systems*, 37:80987–81015, 2024a.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024b.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024c.
- Li, Y., Yuan, P., Feng, S., Pan, B., Wang, X., Sun, B., Wang, H., and Li, K. Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning. *arXiv preprint arXiv:2401.10480*, 2024d.
- Li, Y., Shi, J., Feng, S., Yuan, P., Wang, X., Zhang, Y., Zhang, J., Tan, C., Pan, B., Hu, Y., and Li, K. Speculative decoding for multi-sample inference. *arXiv preprint arXiv:2503.05330*, 2025.
- Liao, B., Xu, Y., Dong, H., Li, J., Monz, C., Savarese, S., Sahoo, D., and Xiong, C. Reward-guided speculative decoding for efficient llm reasoning. *arXiv preprint arXiv:2501.19324*, 2025.
- Luo, X., Wang, Y., Zhu, Q., Zhang, Z., Zhang, X., Yang, Q., Xu, D., and Che, W. Turning trash into treasure: Accelerating inference of large language models with token recycling. *arXiv preprint arXiv:2408.08696*, 2024.
- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Yan, R., Zhu, A., Yang, L., Shi, X., et al. Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification. *arXiv preprint arXiv:2305.09781*, 2023.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., and Hashimoto, T. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- Oliaro, G., Jia, Z., Campos, D., and Qiao, A. Suffixdecoding: A model-free approach to speeding up large language model inference. *arXiv preprint arXiv:2411.04975*, 2024.
- Ou, J., Chen, Y., and Tian, W. Lossless acceleration of large language model via adaptive n-gram parallel decoding. *arXiv preprint arXiv:2404.08698*, 2024.
- Pan, R., Dai, Y., Zhang, Z., Oliaro, G., Jia, Z., and Ne-travali, R. Specreason: Fast and accurate inference-time compute via speculative reasoning. *arXiv preprint arXiv:2504.07891*, 2025.
- Qu, Y., Yang, M. Y., Setlur, A., Tunstall, L., Beeching, E. E., Salakhutdinov, R., and Kumar, A. Optimizing test-time compute via meta reinforcement fine-tuning. *arXiv preprint arXiv:2503.07572*, 2025.

- Saxena, A. Prompt lookup decoding, November 2023. URL <https://github.com/apoorvumang/prompt-lookup-decoding/>.
- Snell, C., Lee, J., Xu, K., and Kumar, A. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Somasundaram, S., Phukan, A., and Saxena, A. Pld+: Accelerating llm inference by leveraging language model artifacts. *arXiv preprint arXiv:2412.01447*, 2024.
- Sui, Y., Chuang, Y.-N., Wang, G., Zhang, J., Zhang, T., Yuan, J., Liu, H., Wen, A., Zhong, S., Chen, H., et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025.
- Team, O. Open Thoughts. <https://open-thoughts.ai>, January 2025.
- Uesato, J., Kushman, N., Kumar, R., Song, F., Siegel, N., Wang, L., Creswell, A., Irving, G., and Higgins, I. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.
- Wan, G., Wu, Y., Chen, J., and Li, S. Dynamic self-consistency: Leveraging reasoning paths for efficient llm sampling. *arXiv preprint arXiv:2408.17017*, 2024.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E. H., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. *ArXiv*, abs/2203.11171, 2022. URL <https://api.semanticscholar.org/CorpusID:247595263>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Xie, Y., Goyal, A., Zheng, W., Kan, M.-Y., Lillicrap, T. P., Kawaguchi, K., and Shieh, M. Monte carlo tree search boosts reasoning via iterative preference learning. *arXiv preprint arXiv:2405.00451*, 2024.
- Yang, W., Yue, X., Chaudhary, V., and Han, X. Speculative thinking: Enhancing small-model reasoning with large model guidance at inference time. *arXiv preprint arXiv:2504.12329*, 2025.

A. Related Works

Test-time scaling and efficiency. Test-Time Scaling (TTS) has emerged as a prominent strategy to enhance problem-solving capabilities during inference without model retraining (Snell et al., 2024; Muennighoff et al., 2025; Wang et al., 2022; Uesato et al., 2022; Xie et al., 2024). Generating long chain-of-thoughts or sampling multiple sequences have consistently showcased higher accuracy in many complex tasks like math, science and coding (Wei et al., 2022; Cobbe et al., 2021; Chen et al., 2023b). However, the computational cost of TTS remains a critical bottleneck for their practical use. Recent work has explored optimizing inference using adaptive thinking lengths, cascading models of different sizes, length penalties during training, and budget-constrained decoding (Aggarwal & Welleck, 2025; Qu et al., 2025; Liao et al., 2025; Li et al., 2024d; Wan et al., 2024), yet the fundamental trade-off between accuracy gains and costs persists. Our method aims to accelerate reasoning while ensuring no performance degradation.

Speculative decoding. SD have been shown to accelerate Large Language Model (LLM) inference without any loss in the accuracy (Leviathan et al., 2023). The approach typically involves a smaller "draft" model proposing candidate token sequences for parallel verification by the larger "target" model. If the tokens align with the target model's output distribution, they are "accepted", resulting in more than one token being produced in a single forward pass of the LLM. Various compute-efficient drafting strategies have been proposed in the literature to increase the chances of acceptance. Neural draft architectures have evolved from simple, smaller LMs to sophisticated self-drafting approaches e.g., Medusa (Cai et al., 2024), Eagle (Li et al., 2024b), and ReDrafter (Cheng et al., 2024)). Although the use of light-weight model-free drafters based on n-grams (Li et al., 2024a; Somasundaram et al., 2024; Hu et al., 2024; Oliaro et al., 2024; Geva et al., 2023; Ou et al., 2024; Saxena, 2023) has been explored previously for generic tasks, we revisit them in the context of LRMs. While these approaches limit themselves to deterministic n-gram based lookups as draft sequences, we highlight the significance of stochastic drafting with logit information of previously generated n-grams for reasoning in our proposed method. To further boost SD performance, tree drafting was proposed where multiple draft token predictions are organized in a tree structure, enabling efficient parallel verification through a specialized tree attention mask (Miao et al., 2023; Li et al., 2024b). Methods like Eagle-2 (Li et al., 2024c) even used dynamic tree layout choices for SD. Extending these existing methods, we additionally propose a computationally efficient data-driven offline tree optimization method for our lightweight model-free drafting method for LRMs.

Other approaches in literature that tie SD with LRMs include Speculative Thinking (Hu et al., 2025), SpecReason (Pan et al., 2025), Reward-guided SD (Liao et al., 2025). However, they do not maintain the lossless nature of SD and hence can also be used in combination with our work. A contemporary work (Li et al., 2025) have explored the importance of model-free n-gram based drafting for multi-sample inference, but did not showcase any practical speedup. We extend their findings with our novel model-free stochastic drafting, and showcase a comparative analysis with existing methods through our extensive experimentation.

B. Motivation

In this section, we highlight the motivation behind each design component of STAND. In Section B.1, we analyze the N-gram overlaps across different reasoning trajectories. The high overlap motivates the use of model-free drafters, which can easily incorporate information from different trajectories. In Section B.2, we analyze the effectiveness of stochastic drafting, compared to the widely-used deterministic drafting strategy. This motivates the sampling-specific components of STAND, including the use of logit-based N-grams, and the Gumbel-Top-K optimization used for speeding up stochastic drafting.

B.1. N-gram overlap analysis

To assess the degree of redundancy in reasoning trajectories, we conducted a comprehensive analysis of n-gram overlap patterns across multiple solutions generated by the DeepSeek-R1-Distill-Qwen-7B model on the AIME-2024 dataset. Figure 3 illustrates our findings, depicting the overlap rates for n-grams ranging from bigrams to 5-grams across varying numbers of reasoning trajectories.

The results reveal a substantial level of repetition in token sequences. Notably, we observed that up to 97% of bigrams and 80% of 4-grams recur across 16 distinct reasoning trajectories. Even when considering only two trajectories, over 90% of bigrams are repeated. This high degree of overlap suggests a significant probability that any given n-gram generated by the model has likely appeared in a previous trajectory.

These findings present a compelling opportunity for the development of an efficient drafting strategy. By leveraging this inherent redundancy, we can implement a straightforward approach where previously generated n-grams are proposed as draft sequences, potentially leading to significant improvements in computational efficiency without compromising the chance of acceptance of the generated draft. This presents a key motivation for our proposed method STAND.

B.2. Effectiveness of stochastic drafting

In contrast to traditional generation approaches that rely on greedy decoding, LRMs typically employ sampling-based generation strategies to produce multiple diverse solution trajectories, making the choice of drafting strategy particularly crucial. In speculative sampling (Chen et al., 2023a), given a target distribution $p(x)$ and draft distribution $q(x)$, the speculative sampling procedure operates by first sampling $x \sim q(x)$. The sampled token is accepted if $q(x) \leq p(x)$. Otherwise, when $q(x) > p(x)$, the token is rejected with probability $1 - \frac{p(x)}{q(x)}$ and resampled from an adjusted distribution $p'(x) = \text{norm}(\max(0, p(x) - q(x)))$. This procedure guarantees that the final output distribution matches the target distribution $p(x)$, for any drafting distribution $q(x)$.

One can choose the drafting strategy to be deterministic or stochastic. In the former, $q(x)$ is treated as a one-hot vector where $q(x_{\text{draft}}) = 1$ for the drafted token x_{draft} and $q(x) = 0$ for all other x . For speculative sampling, this means the drafted token is accepted with $p(x_{\text{draft}})$, which can be particularly low when the target model is uncertain about its prediction. In contrast, stochastic drafting generates drafts through sampling from a probability distribution. Aligning this draft distribution with the target can significantly boost the chances of acceptance.

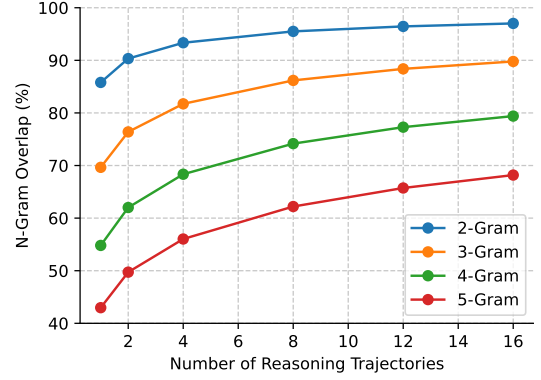


Figure 3. **N-gram overlaps across reasoning trajectories.** We report the N-gram overlaps across different number of reasoning trajectories, generated by DeepSeek-R1-Distill-Qwen-7B on AIME-2024. The overlap is defined as the percentage of the N-grams that appear twice or more in k reasoning trajectories, counting duplicates multiple times.

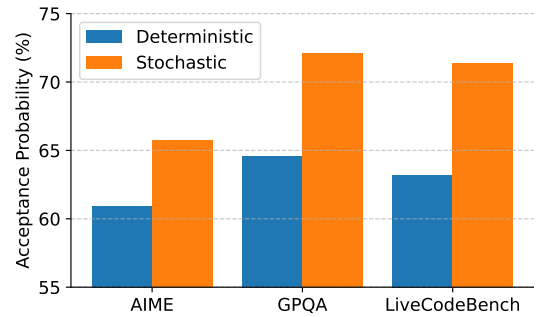


Figure 4. **Deterministic vs. stochastic drafting.** We report the acceptance probability of a token, given a draft tree with depth 1 and width 3. Measurements are done using DeepSeek-R1-Distill-Qwen-7B model, and the draft tree is constructed using the N-gram module in STAND.

In generic greedy decoding setups where this choice does not matter, existing model-free SD methods (Ou et al., 2024; Hu et al., 2024; Saxena, 2023) do not store any probability distribution with the n-gram lookup-based drafters. Eagle-2 (Li et al., 2024c) also uses deterministic drafting for better compatibility with their dynamic tree construction logic. However, for LRMs where sampling plays a key role in generation, we showcase that this choice plays a pivotal role in acceptance probability of the draft sequence. As shown in Figure 4, this fundamental difference leads to 5%, 7% and 8% higher acceptance probabilities for stochastic drafting compared to deterministic drafting across different reasoning tasks i.e. AIME, GPQA and LiveCodeBench respectively. These experimental findings motivated us to find effective ways to compute draft model probabilities in STAND, that aligns well with the probability distributions of LRMs from which the multiple trajectories are sampled.

C. Experimental Details and Additional Results

Experimental setup and baselines. Throughout the experiments, we evaluate the effectiveness of our approach on diverse tasks, including math reasoning (AIME-2024), STEM QA (GPQA-Diamond), and coding (LiveCodeBench). We perform evaluations across different model scales, including DeepSeek-R1-Distill-Qwen-7B and 14B. For all tasks, we generate maximum 32k tokens for the 7B model, and 24k tokens for the 14B model. All sampling is done with temperature 0.6. All measurements are done on a single A100 GPU, with 30 samples per task for efficient experiments.

For model-free baselines, we compare STAND against Prompt Lookup Decoding (PLD, (Saxena, 2023)), Adaptive N-gram Parallel Decoding (ANPD, (Ou et al., 2024)), Token Recycle (Recycle, (Luo et al., 2024)), SAM-Decoding (SAM, (Hu et al., 2024)) and a combination of SAM decoding and Token Recycle, also proposed in the SAM paper. For Static SAM (which is a component of SAM that uses a pre-constructed suffix automation from a datastore), we construct the datastore using 4k samples from the OpenThoughts-114k (Team, 2025) dataset. We also compare against Eagle-2, as a representative model-based baseline. To enable long context inference, we trained all Eagle models using OpenThoughts-114k dataset, where long samples exceeding 32k tokens were truncated.

Evaluation metrics. We adopt throughput and acceptance length as the main evaluation metrics. Throughput is obtained by dividing the total drafting time with the number of generated tokens. Acceptance length describes how many tokens were accepted per speculation step. Higher values are better for both metrics.

C.1. Extended main table

In Table 2, we present the extended main table with additional baselines and extended model scale. STAND consistently outperforms all baselines, both in throughput and acceptance lengths.

Table 2. Speculative decoding performance in multi-trajectory reasoning. We report the average throughput (T) and acceptance length (A) for multi-trajectory test-time scaling scenarios, with different number of reasoning trajectories per problem. We evaluate each model on AIME-2024 (AIME), GPQA-Diamond (GPQA), and LiveCodeBench (LCB). The best values are highlighted in **bold**.

		4 Trajectories			8 Trajectories			16 Trajectories		
		AIME	GPQA	LCB	AIME	GPQA	LCB	AIME	GPQA	LCB
<i>DeepSeek-R1-Distill-Qwen-7B</i>										
Plain	T	26.63	31.34	27.75	26.63	31.34	27.75	26.63	31.34	27.75
	A	2.21	1.99	2.13	2.21	1.99	2.13	2.21	1.99	2.13
Eagle-2	T	29.91	31.69	27.61	29.91	31.69	27.61	29.91	31.69	27.61
	A	2.21	1.99	2.13	2.21	1.99	2.13	2.21	1.99	2.13
PLD	T	43.93	50.49	44.01	44.95	53.04	45.08	46.60	53.47	46.02
	A	1.78	1.81	1.73	1.84	1.89	1.79	1.89	1.96	1.85
ANPD	T	45.52	57.39	46.30	46.40	58.97	47.86	47.06	60.25	48.81
	A	1.89	1.97	1.88	1.92	2.03	1.91	1.96	2.11	1.96
SAM	T	44.35	53.21	45.63	45.64	55.47	47.24	47.64	57.53	48.92
	A	1.81	1.87	1.85	1.89	1.96	1.89	1.97	2.03	1.95
Recycle	T	61.38	71.51	60.62	61.70	71.55	60.93	60.86	71.23	61.36
	A	2.76	2.73	2.73	2.77	2.73	2.73	2.77	2.73	2.74
SAM + Recycle	T	61.11	70.43	62.20	60.66	69.98	63.41	60.63	69.85	63.39
	A	2.71	2.73	2.68	2.69	2.74	2.69	2.68	2.71	2.67
STAND (Ours)	T	64.99	83.47	69.70	66.88	87.02	71.83	69.15	91.17	74.14
	A	3.21	3.48	3.30	3.35	3.70	3.47	3.46	3.90	3.64
<i>DeepSeek-R1-Distill-Qwen-14B</i>										
Plain	T	17.76	18.16	17.43	17.76	18.16	17.43	17.76	18.16	17.43
	A	2.72	2.44	2.51	2.72	2.44	2.51	2.72	2.44	2.51
Eagle-2	T	25.38	24.86	21.89	25.38	24.86	21.89	25.38	24.86	21.89
	A	2.72	2.44	2.51	2.72	2.44	2.51	2.72	2.44	2.51
PLD	T	24.37	26.60	23.36	25.44	27.36	23.96	26.35	28.43	24.97
	A	1.74	1.82	1.74	1.84	1.91	1.81	1.92	2.00	1.88
ANPD	T	25.74	28.21	24.78	26.12	29.51	25.63	26.49	30.62	26.32
	A	1.87	1.97	1.87	1.91	2.04	1.93	1.96	2.13	1.99
SAM	T	25.22	28.03	24.41	26.11	29.39	25.37	27.25	30.59	26.67
	A	1.78	1.85	1.79	1.88	1.95	1.87	1.98	2.06	1.96
Recycle	T	34.97	38.99	34.05	35.06	38.89	33.98	35.31	38.81	33.96
	A	2.78	2.73	2.72	2.77	2.73	2.72	2.77	2.74	2.72
SAM + Recycle	T	34.81	38.24	34.15	35.16	38.57	34.19	35.53	38.99	34.31
	A	2.70	2.71	2.65	2.71	2.71	2.66	2.72	2.71	2.65
STAND (Ours)	T	37.56	43.71	38.71	39.13	46.81	40.45	40.76	49.11	42.72
	A	3.16	3.42	3.29	3.28	3.63	3.47	3.42	3.86	3.65

C.2. Evaluation on single-trajectory decoding

Table 3. **Single-trajectory evaluations.** We report the throughput (T) and acceptance length (A) for generating a single sequence with DeepSeek-R1-Distill-Qwen-7B and 14B. The best values are highlighted in **bold**, and the runner-up is underlined.

	AIME		GPQA		LCB	
	T	A	T	A	T	A
<i>DeepSeek-R1-Distill-Qwen-7B</i>						
Plain	26.63	x	31.34	x	27.75	x
Eagle-2	29.91	2.21	31.69	1.99	27.61	2.13
PLD	44.34	1.72	42.84	1.64	43.40	1.59
ANPD	46.18	1.88	54.05	1.82	44.79	1.80
SAM	40.85	1.69	48.45	1.69	42.92	1.80
Recycle	60.61	<u>2.73</u>	71.00	2.71	60.12	<u>2.73</u>
SAM + Recycle	61.15	<u>2.70</u>	71.51	2.81	62.78	<u>2.69</u>
Ours	61.79	3.07	75.39	3.05	66.41	3.01
<i>DeepSeek-R1-Distill-Qwen-14B</i>						
Plain	17.76	x	18.16	x	17.43	x
Eagle-2	25.38	2.72	24.86	2.44	21.89	2.51
PLD	21.82	1.61	24.97	1.64	21.76	1.58
ANPD	25.60	1.76	26.40	1.79	23.16	1.76
SAM	23.26	1.63	25.38	1.65	22.36	1.63
Recycle	33.71	<u>2.77</u>	38.91	2.73	33.85	<u>2.71</u>
SAM + Recycle	34.35	<u>2.67</u>	37.53	<u>2.72</u>	34.45	<u>2.70</u>
Ours	34.52	2.91	<u>38.71</u>	3.00	34.86	2.93

While STAND is primarily designed to leverage information across multiple reasoning trajectories, we also evaluate its performance on single-trajectory generation, where the model only produces one long reasoning chain. As shown in Table 3, STAND achieves both the highest acceptance length and throughput in most scenarios, demonstrating its effectiveness even when generating individual solutions.

C.3. Ablations and analysis

We evaluate key components of STAND through ablation studies and further analysis. Our ablation studies examine the impact of stochastic drafting and the Gumbel-Top-K optimization trick, followed by an investigation of our tree optimization approach. We then analyze the structural characteristics of the optimized trees to better understand the patterns that emerge from our method.

Effect of stochastic drafting. In Table 4, we compare three drafting approaches: deterministic drafting, basic stochastic drafting (using PyTorch’s multinomial sampling), and our optimized stochastic drafting with Gumbel-Top-K. For fair comparison, we separately perform tree optimization for deterministic drafting and stochastic drafting. Stochastic drafting consistently achieves higher acceptance lengths across all tasks, resulting in improved throughput compared to deterministic drafting. Our Gumbel-Top-K optimization further improves performance by maintaining similar acceptance lengths while significantly reducing latency, leading to even higher throughput.

Table 4. **Effect of Stochastic Drafting.** We report the throughput for generating 4 sequences with DeepSeek-R1-Distill-Qwen-7B, on AIME-2024.

	AIME		GPQA		LCB	
	T	A	T	A	T	A
Deterministic	62.13	2.94	73.67	2.98	63.44	2.90
Stochastic	63.44	3.24	81.20	3.56	65.90	3.29
+ Gumbel-Top-K	64.99	3.21	83.47	3.48	69.70	3.30

Effect of tree optimization. In Table 5, we showcase the effectiveness of our tree optimization technique. Specifically, we compare the performance of a heuristic tree originally used by Token Recycle (Luo et al., 2024) with our tree, optimized

on the AIME-2024 dataset. The results demonstrates that the optimized tree improves performance on both AIME-2024 and GPQA-Diamond, showcasing that the optimization not only works for the dataset in the same domain, but also generalizes to out-of-domain (OOD) tasks.

Table 5. Effect of tree optimization. Comparison of mean acceptance lengths when generating 4 sequences with DeepSeek-R1-Distill-Qwen-7B on two datasets: AIME-2024 and GPQA-Diamond. We compare two types of static trees: the heuristic trees from Token Recycle and our data-optimized trees (optimized on AIME-2024). AIME-2024 represents in-domain (IND) performance since it was used for tree optimization, while GPQA-Diamond tests out-of-domain (OOD) generalization. The best values are highlighted in **bold**.

	AIME (IND)		GPQA (OOD)	
	Heuristic	Optimized	Heuristic	Optimized
Throughput	59.96	64.99	77.32	83.47
Acc. Lens	3.17	3.21	3.35	3.48

Tree structure analysis. We analyze how different drafting approaches lead to different optimal tree structures by comparing trees optimized for STAND versus Token Recycle. As shown in Figure 5a, the tree optimized for STAND reaches greater depths, extending to 13 levels compared to 7 levels in the Token Recycle-optimized tree. This difference likely stems from STAND’s higher acceptance rate, which favors deeper, narrower tree structures under the same tree size budget.

A distinctive feature of STAND’s optimized tree is its long tail structure, with single nodes at depths 8 through 13. This pattern suggests the presence of occasional long, deterministic sequences, possibly arising from consistent patterns found across multiple reasoning trajectories.

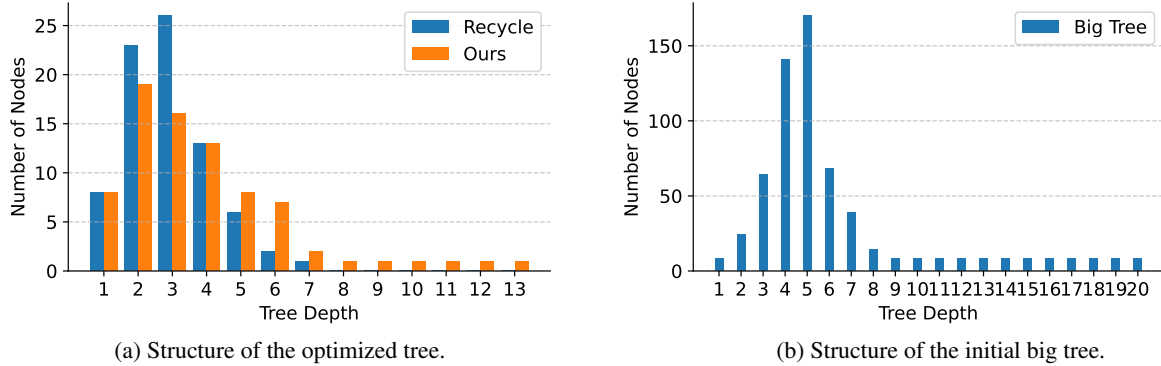


Figure 5. Tree structure analysis. (Left) We report the number of nodes at specific tree depths for draft trees optimized for each Token Recycle and STAND. Both trees are optimized on AIME-2024 dataset with DeepSeek-R1-Distill-Qwen-7B. (Right) We report the number of nodes at specific tree depths for the initial tree used for tree optimization.

Table 6. DVTS.

		AIME	GPQA	LCB
Plain	T	33.35	33.12	32.22
	A	2.73	2.69	2.74
Recycle	T	71.52	70.64	69.47
	A	2.73	2.69	2.74
STAND (Ours)	T	82.62	86.93	80.97
	A	3.55	3.71	3.51

Table 7. **Batch.**

Batch Size		AIME		GPQA		LCB	
		4	8	4	8	4	8
Plain	T	89.04	111.62	88.32	106.23	92.64	114.73
	A	1.77	1.66	1.83	1.75	1.72	1.63
Recycle	T	90.51	99.18	98.41	105.69	89.55	99.20
	A	1.77	1.66	1.83	1.75	1.72	1.63
STAND (Ours)	T	127.58	148.08	134.64	154.68	122.09	149.41
	A	2.57	2.70	2.74	2.86	2.57	2.69